

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 02

Total Marks:20

Annexure A

Experiments

Code of Conduct:

I Abinesh H (192424387) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Criteria	Conceptual Understanding	Accuracy of Answers	Application of Knowledge	Completeness of Response	Clarity & Presentation	Total
Weightage	30%	25%	15%	15%	15%	
Q1						
Q2						

Signature of the student : Abinesh H

Total Marks :

Short Answer Test

1, Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

Experiment 1: Implementation and Evaluation of a Naïve Bayes Classifier

1. Aim

To design and conduct an experiment implementing a **Naïve Bayes classifier** on a real-world dataset, train the classifier using different feature sets, evaluate its performance using **accuracy, precision, and recall**, analyze the effect of the feature-independence assumption, and compare classification performance with and without **Laplace smoothing**.

2. Objectives

1. To understand the working principle of the Naïve Bayes classification algorithm.
 2. To implement a Naïve Bayes classifier using a real-world dataset.
 3. To experiment with different feature sets.
 4. To evaluate the classifier using accuracy, precision, and recall.
 5. To study the effect of the assumption that features are conditionally independent.
 6. To compare Naïve Bayes with and without Laplace smoothing.
 7. To understand how probability estimation affects classification performance.
-

3. Theory

3.1 Naïve Bayes Classifier

Naïve Bayes is a probabilistic classification algorithm based on **Bayes' theorem**.

For a class (C) and feature vector ($X=(x_1, x_2, \dots, x_n)$),

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

Since ($P(X)$) is the same for all classes, classification is performed using:

$$P(C|X) \propto P(C)P(X|C)$$

The "naïve" assumption is that the features are **conditionally independent given the class**:

$$P(X|C) = \prod_{i=1}^n P(x_i|C)$$

Therefore,

$$P(C|X) \propto P(C) \prod_{i=1}^n P(x_i|C)$$

The classifier assigns the observation to the class having the highest posterior probability.

4. Dataset

SMS Spam Collection Dataset

The **SMS Spam Collection** is a real-world dataset containing SMS messages labeled as:

- **ham** – legitimate message
- **spam** – unwanted/spam message

Text classification is a suitable application for Naïve Bayes because a message can be represented using word features and the algorithm can estimate the probability of each class from the occurrence of those words.

For this experiment, two feature sets are considered:

Feature Set 1 – Unigrams

Only individual words are used as features.

Example:

"Win a free prize"

Features:

- win
- a
- free
- prize

Feature Set 2 – Unigrams + Bigrams

Both individual words and pairs of consecutive words are used.

Example:

"Win a free prize"

Possible features include:

- win
- free
- prize
- win a
- a free
- free prize

The second feature set provides more contextual information than unigrams alone.

5. Algorithm

Naïve Bayes Classification

Step 1: Load the SMS dataset.

Step 2: Convert the class labels into numerical labels.

Step 3: Split the dataset into training and testing sets.

Step 4: Convert SMS text into numerical feature vectors using TF-IDF.

Step 5: Train a Multinomial Naïve Bayes classifier.

Step 6: Repeat the experiment using:

- Unigram features.
- Unigram + bigram features.

Step 7: Evaluate each model using:

- Accuracy
- Precision
- Recall

Step 8: Repeat the experiment with different values of the smoothing parameter (α).

Step 9: Compare the results.

6. Python Implementation

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score,
classification_report

# -----
# 1. Load the SMS Spam Collection dataset
# -----

url = "https://raw.githubusercontent.com/justmarkham/pycon-2016-
tutorial/master/data/sms.tsv"

data = pd.read_csv(url, sep="\t", header=None, names=["label", "message"])

print("Dataset shape:", data.shape)
print(data.head())

# -----
# 2. Convert labels
# ham = 0
# spam = 1
# -----

data["label_num"] = data["label"].map({"ham": 0, "spam": 1})

X = data["message"]
y = data["label_num"]

# -----
# 3. Split into training and testing data
# -----
```

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# -----
# 4. Function for training and evaluation
# -----

def run_experiment(ngram_range, alpha):

    vectorizer = TfidfVectorizer(
        ngram_range=ngram_range,
        lowercase=True,
        stop_words="english"
    )

    X_train_vec = vectorizer.fit_transform(X_train)
    X_test_vec = vectorizer.transform(X_test)

    model = MultinomialNB(alpha=alpha)

    model.fit(X_train_vec, y_train)

    y_pred = model.predict(X_test_vec)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, zero_division=0)
    recall = recall_score(y_test, y_pred, zero_division=0)

    return accuracy, precision, recall

# -----
# 5. Experiment with different feature sets
# -----

# Feature Set 1: Unigrams
uni_results = run_experiment((1, 1), alpha=1.0)

# Feature Set 2: Unigrams + Bigrams
uni_bi_results = run_experiment((1, 2), alpha=1.0)

print("\nResults with Laplace smoothing:")
print("Unigrams:")
print("Accuracy :", uni_results[0])
print("Precision:", uni_results[1])

```

```

print("Recall  :", uni_results[2])

print("\nUnigrams + Bigrams:")
print("Accuracy :", uni_bi_results[0])
print("Precision:", uni_bi_results[1])
print("Recall  :", uni_bi_results[2])

# -----
# 6. Compare different smoothing values
# -----

alphas = [0, 0.01, 0.1, 1.0, 10.0]

print("\nSmoothing Experiment")

for alpha in alphas:

    # MultinomialNB does not permit exactly zero alpha
    # in all versions, so alpha=0 is handled separately.
    if alpha == 0:
        alpha_used = 1e-10
    else:
        alpha_used = alpha

    accuracy, precision, recall = run_experiment(
        (1, 1),
        alpha_used
    )

    print(
        f"Alpha={alpha}: "
        f"Accuracy={accuracy:.4f}, "
        f"Precision={precision:.4f}, "
        f"Recall={recall:.4f}"
    )

# -----
# 7. Detailed classification report
# -----

vectorizer = TfidfVectorizer(
    ngram_range=(1, 2),
    lowercase=True,
    stop_words="english"
)

X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

```

```

model = MultinomialNB(alpha=1.0)
model.fit(X_train_vec, y_train)

y_pred = model.predict(X_test_vec)

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=["Ham", "Spam"]
    )
)

```

7. Evaluation Metrics

The following metrics are used to evaluate the classifier.

Accuracy

Accuracy measures the proportion of correctly classified messages.

```

[
Accuracy=
\frac{TP+TN}{TP+TN+FP+FN}
]

```

where:

- (TP) = True Positives
 - (TN) = True Negatives
 - (FP) = False Positives
 - (FN) = False Negatives
-

Precision

Precision measures how many messages predicted as spam are actually spam.

```

[
Precision=\frac{TP}{TP+FP}
]

```

High precision means that legitimate messages are rarely incorrectly classified as spam.

Recall

Recall measures how many actual spam messages are correctly identified.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

High recall means that fewer spam messages are missed.

8. Experimental Results

A typical result table can be recorded in the following format after executing the program.

Feature Set	Laplace Smoothing	Accuracy	Precision	Recall
Unigrams	($\alpha=1$)	~0.96– 0.98	~0.95– 0.98	~0.85– 0.92
Unigrams + Bigrams	($\alpha=1$)	~0.96– 0.99	~0.95– 0.99	~0.88– 0.95

Note: The exact values may vary depending on the dataset version, preprocessing, train-test split, vectorization settings, and random seed. The values above are representative ranges rather than fabricated exact experimental measurements. The exact output generated by the code should be entered into the final results table.

9. Experiment: Effect of Laplace Smoothing

Without Smoothing

For a word (w), the conditional probability can be estimated as:

$$P(w|C) = \frac{\text{count}(w, C)}{\sum_{w'} \text{count}(w', C)}$$

If a word does not occur in the training examples of a particular class, its probability becomes zero:

$$[P(w|C)=0]$$

Because Naïve Bayes multiplies the probabilities of all features, one zero probability can make the complete probability zero.

This is known as the **zero-frequency problem**.

With Laplace Smoothing

Laplace smoothing adds 1 to every feature count:

$$[P(w|C)=\frac{\text{count}(w,C)+1}{\sum_{w'}\text{count}(w',C)+V}]$$

where (V) is the vocabulary size.

More generally, the smoothing parameter (α) gives:

$$[P(w|C)=\frac{\text{count}(w,C)+\alpha}{\sum_{w'}\text{count}(w',C)+\alpha V}]$$

For:

$$[\alpha=1]$$

this is standard Laplace smoothing.

10. Typical Smoothing Results

The experiment can be summarized as follows:

Alpha	Smoothing	Expected Behaviour
-------	-----------	--------------------

0	No smoothing	Sensitive to unseen features
0.01	Very small smoothing	Reduces zero-probability problem
0.1	Small smoothing	Usually stable
1.0	Laplace smoothing	Good general-purpose setting
10.0	Strong smoothing	May oversmooth the probabilities

The best value of (α) is data-dependent. Increasing (α) prevents extreme probabilities, but excessively large values can make the probability estimates too uniform and reduce the classifier's ability to distinguish classes.

11. Analysis of Feature Independence Assumption

The fundamental assumption of Naïve Bayes is:

[

$$P(x_1, x_2, \dots, x_n | C)$$

$$\prod_i P(x_i | C)$$

This means that features are assumed to be conditionally independent once the class is known.

For SMS messages, this assumption is not completely true.

For example:

"free prize"

The words **free** and **prize** are related in the context of spam messages. Similarly:

"call me"

contains a relationship between the two words.

Therefore, treating every word as completely independent is an approximation.

Effect of the assumption

Despite this unrealistic assumption, Naïve Bayes can perform very well on text classification.

This occurs because the classifier does not necessarily need perfectly accurate probability estimates to make correct classifications. It mainly needs the correct class to have the largest posterior probability.

The use of bigrams partially captures relationships between neighboring words.

For example:

Unigram model:

- free
- prize

Unigram + bigram model:

- free
- prize
- free prize

The bigram "**free prize**" contains more contextual information and can therefore improve discrimination between spam and legitimate messages.

However, adding more features can also increase the dimensionality of the feature space and may introduce redundant or correlated features.

12. Interpretation of Probability Estimation

Naïve Bayes relies heavily on estimated probabilities.

Suppose a word such as "**winner**" occurs frequently in spam messages but rarely in legitimate messages.

The classifier estimates:

[
 $P(\text{winner}|\text{spam})$
]

and

[
 $P(\text{winner}|\text{ham})$
]

If the first probability is much larger than the second, the presence of the word increases the posterior probability of the spam class.

However, probabilities estimated from a small number of observations can be unreliable.

For example, if a word occurs only once in the training set, its estimated probability can change significantly depending on whether that single occurrence belongs to spam or ham.

Laplace smoothing reduces this sensitivity by assigning a small non-zero probability to unseen or rare features.

Therefore:

- **No smoothing:** probability estimates can become zero or extreme.
 - **Moderate smoothing:** probability estimates become more robust.
 - **Excessive smoothing:** differences between classes can become too small.
-

13. Comparison of Feature Sets

Unigram Features

Advantages:

- Simple representation.
- Lower computational complexity.
- Works well for text classification.
- Easy to interpret.

Disadvantages:

- Ignores word order.
- Does not capture short phrases.
- May lose contextual information.

Unigram + Bigram Features

Advantages:

- Captures short word combinations.
- Provides more contextual information.

- Can improve spam detection for characteristic phrases.

Disadvantages:

- Produces more features.
 - Requires more memory and computation.
 - Some bigrams may be rare.
 - More correlated features can weaken the independence assumption.
-

14. Discussion

The experiment demonstrates that Naïve Bayes is an effective classifier for real-world text data despite its strong independence assumption.

The unigram model provides a strong baseline because individual words often carry substantial information about whether an SMS is spam or legitimate.

Adding bigrams can improve classification because phrases provide contextual information that individual words cannot represent. However, improvement is not guaranteed because the additional features increase dimensionality and may be correlated.

Laplace smoothing is particularly important when the training set contains words that are absent from one of the classes. Without smoothing, a zero probability can dominate the complete posterior probability. Smoothing prevents this problem by assigning a small non-zero probability to unseen features.

The experiment also demonstrates that probability estimation and classification performance are related but not identical. A model may have imperfect probability estimates but still achieve high classification accuracy if it consistently ranks the correct class above the incorrect class.

15. Advantages of Naïve Bayes

1. Simple and easy to implement.
 2. Fast training and prediction.
 3. Performs well with high-dimensional text data.
 4. Requires relatively little training data.
 5. Can work effectively even when the independence assumption is not perfectly true.
 6. Laplace smoothing provides a simple solution to zero-frequency problems.
-

16. Limitations

1. The feature independence assumption is often unrealistic.
 2. Correlated features may result in inaccurate probability estimates.
 3. Probability estimates may be poorly calibrated.
 4. Rare features can cause unstable estimates without smoothing.
 5. Strongly dependent features may be counted multiple times.
-

17. Conclusion

A Naïve Bayes classifier was successfully implemented on the real-world SMS Spam Collection dataset. Two feature representations, **unigrams** and **unigrams combined with bigrams**, were investigated and evaluated using accuracy, precision, and recall.

The experiment shows that Naïve Bayes can provide high classification performance even though it assumes conditional independence between features. Adding bigram features can provide additional contextual information, although the resulting features may be correlated and can increase computational complexity.

The comparison of smoothing methods demonstrates the importance of **Laplace smoothing** in probability estimation. Without smoothing, unseen features can produce zero probabilities and adversely affect classification. With an appropriate smoothing value, the model becomes more robust and reliable.

Overall, the experiment demonstrates that Naïve Bayes is a computationally efficient and effective algorithm for text classification, while the choice of feature representation and probability-smoothing strategy has an important effect on its final performance.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Output:

Dataset shape: (5572, 2)

	label	message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Results with Laplace smoothing:

Unigrams:

Accuracy : 0.9704035874439462

Precision: 1.0

Recall : 0.7785234899328859

Unigrams + Bigrams:

Accuracy : 0.9623318385650225

Precision: 1.0

Recall : 0.7181208053691275

Smoothing Experiment

Alpha=0: Accuracy=0.9731, Precision=0.9281, Recall=0.8658

Alpha=0.01: Accuracy=0.9812, Precision=0.9638, Recall=0.8926

Alpha=0.1: Accuracy=0.9839, Precision=0.9781, Recall=0.8993

Alpha=1.0: Accuracy=0.9704, Precision=1.0000, Recall=0.7785

Alpha=10.0: Accuracy=0.8673, Precision=1.0000, Recall=0.0067

Classification Report:

	precision	recall	f1-score	support
Ham	0.96	1.00	0.98	966
Spam	1.00	0.72	0.84	149
accuracy		0.96		1115
macro avg	0.98	0.86	0.91	1115
weighted avg	0.96	0.96	0.96	1115